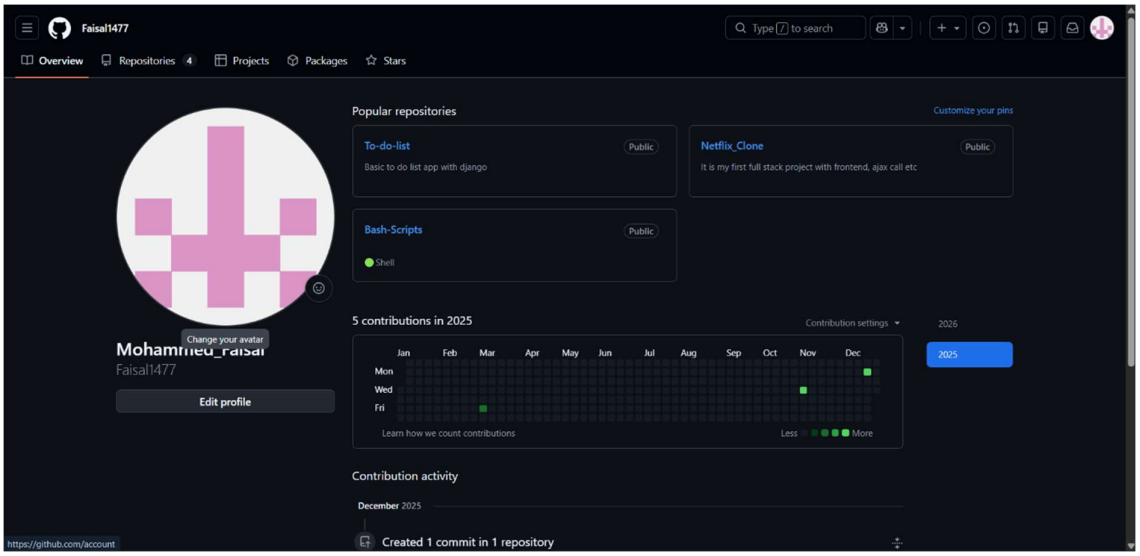
Git & GitHub Task

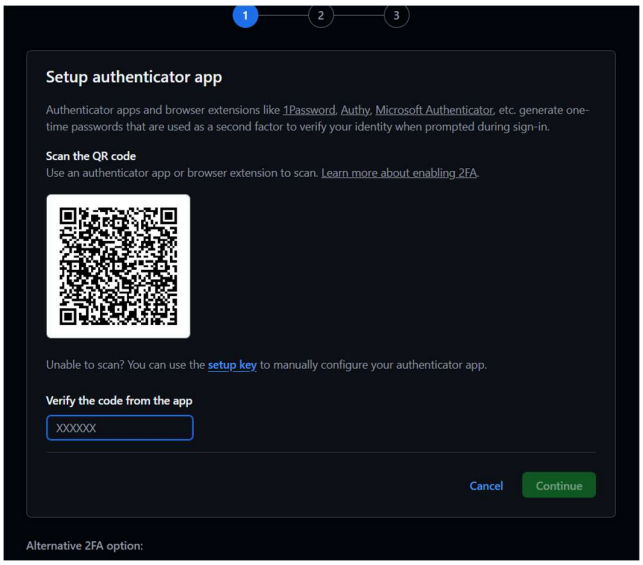
Create a GitHub Account



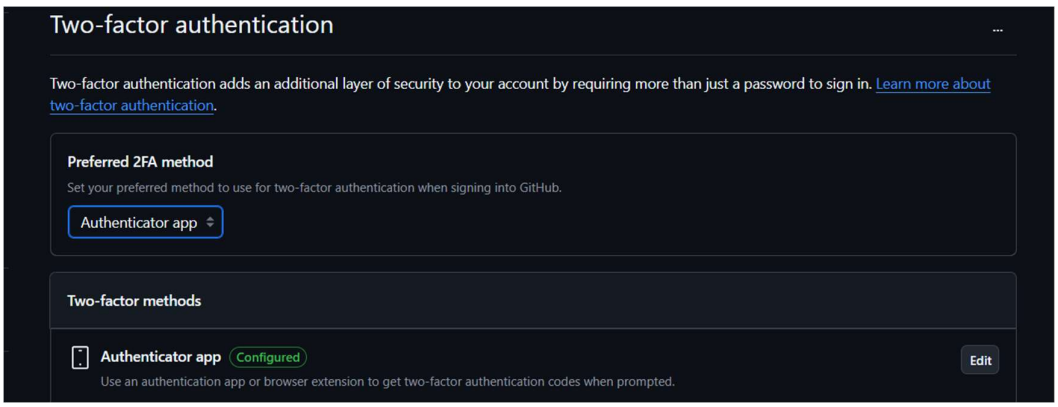
Enable MFA (Multi factor Authentication)

Go to settings and search for two factor authentication. Then create authentication

In first step we have to scan Qr code in google authenticator and enter the pin



In second step download the recovery keys file. Then the setup is completed

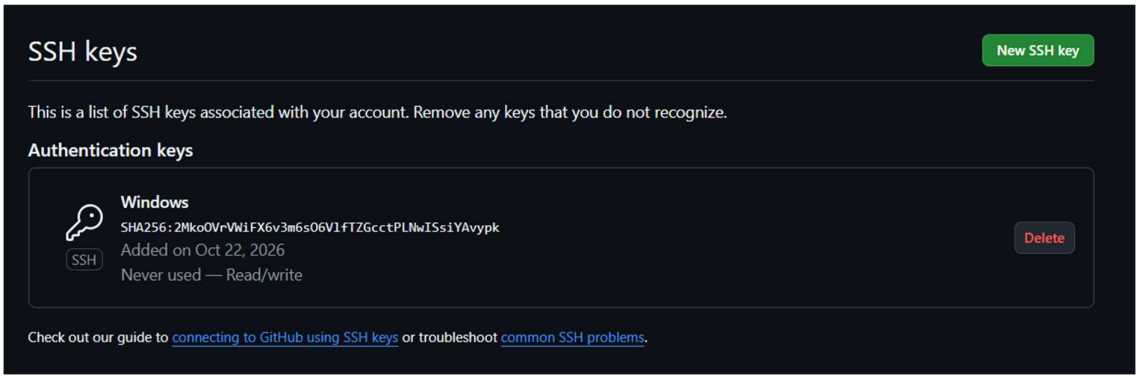


Now generate a ssh key for authentication in gitbash.

```
Faisal@LAPTOP-4DO8SS4E MINGW64 /d/Git (main)
$ ssh-keygen
Generating public/private rsa key pair.
Enter file in which to save the key (/c/Users/SUBHAN/.ssh/id_rsa):
/c/Users/SUBHAN/.ssh/id_rsa already exists.
Overwrite (y/n)? y
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /c/Users/SUBHAN/.ssh/id_rsa
Your public key has been saved in /c/Users/SUBHAN/.ssh/id_rsa.pub
The key fingerprint is:
SHA256:2tzIhZz/rUsPQfXLL76SPproapuo2x2uUkj7ZQ1yibw Faisal@LAPTOP-4DO8SS4E
The key's randomart image is:
+---[RSA 3072]----+
|             .   |
|       . o   . . |
|   . . .+ . .   .|
|  . + +o = .  . .|
| . o + oS o .  o |
|  o E oo.  . o  .|
|   o o.     +.o .|
|  ...+.o . o+= . |
|  o++o*+o oo+++. |
+----[SHA256]-----+

Faisal@LAPTOP-4DO8SS4E MINGW64 /d/Git (main)
$ cat /c/Users/SUBHAN/.ssh/id_rsa.pub
ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAABgQDtrGsXM4AmXM3j0esEjEl2mGgLwNL7yOanrcHG8Vq8p/bdg3gKF
YIgbq5pOJTDGkKBPIal8Wee35V+g2HknaapD3KAkCwnrmQmx+9QIjdbHMkaiXPz3KFMX5LO1geqYpbhQvNqGlgOvV
fp5NNjePowspIvN3BRPv+ULq5/49aKUxRatv+H/Q9mPga54sIPxtFDStKvQdgxrpBHCn0cq5HakMg9rkODe+lhdIC
```

Then configure it in the ssh key settings.



Git Configurations:- We have to configure username and email id which will be reflected in name of author

```
Faisal@LAPTOP-4DO8SS4E MINGW64 /d/Git
$ git config --global user.name "Faisal1477"

Faisal@LAPTOP-4DO8SS4E MINGW64 /d/Git
$ git config --global user.email "mohdfaisal1477@gmail.com"
```

We can see the available configuration with --list

```
Faisal@LAPTOP-4DO8SS4E MINGW64 /d/Git
$ git config --list
diff.astextplain.textconv=astextplain
filter.lfs.clean=git-lfs clean -- %f
filter.lfs.smudge=git-lfs smudge -- %f
filter.lfs.process=git-lfs filter-process
filter.lfs.required=true
http.sslbackend=schannel
core.autocrlf=true
core.fscache=true
core.symlinks=false
pull.rebase=false
credential.helper=manager-core
credential.https://dev.azure.com.usehttppath=true
init.defaultbranch=master
user.name=Faisal1477
user.email=mohdfaisal1477@gmail.com
filter.lfs.smudge=git-lfs smudge -- %f
filter.lfs.process=git-lfs filter-process
filter.lfs.required=true
filter.lfs.clean=git-lfs clean -- %f
init.defaultbranch=main
```

Git Init:- Initializing or creating a local Repository and we can see that new folder is added with ./git.

Git Status:- It can be used to check the status of files whether they are staged or not and whether the commit has been done or not. It mainly helps us when changes are made.

```
Faisal@LAPTOP-4DO8SS4E MINGW64 /d/Git
$ git init
Initialized empty Git repository in D:/Git/.git/

Faisal@LAPTOP-4DO8SS4E MINGW64 /d/Git (main)
$ git status
On branch main

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        text.txt

nothing added to commit but untracked files present (use "git add" to track)
```

Git add:- It is used to add files to local repo. We can add single file with name and use '.' to add files.

```
Faisal@LAPTOP-4DO8SS4E MINGW64 /d/Git (main)
$ git add .

Faisal@LAPTOP-4DO8SS4E MINGW64 /d/Git (main)
$ git status
On branch main

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   Git.txt
        new file:   greet
        new file:   hello
        new file:   hi
```

Git commit :- Finalizing the changes and writing the message with -m for the changes done.

```
Faisal@LAPTOP-4DO8SS4E MINGW64 /d/Git (main)
$ git commit -m "Initial setup"
[main (root-commit) 6772b06] Initial setup
 4 files changed, 1 insertion(+)
 create mode 100644 Git.txt
 create mode 100644 greet
 create mode 100644 hello
 create mode 100644 hi

Faisal@LAPTOP-4DO8SS4E MINGW64 /d/Git (main)
$ git status
On branch main
nothing to commit, working tree clean
```

Git Log:- We need to remember the id of commit if we want to revert back we can check it with Git log

```
Faisal@LAPTOP-4DO8SS4E MINGW64 /d/Git (main)
$ git log
commit 6772b069214ebedebbfcb2bc85a199cf1df76124 (HEAD -> main, origin/main)
Author: Faisal1477 <mohdfaisal1477@gmail.com>
Date:   Thu Oct 22 16:17:18 2026 +0530

    Initial setup
```

Git remote :- remote command has many uses to add, Check, remove Repo Url. This url tells us where to upload the files.

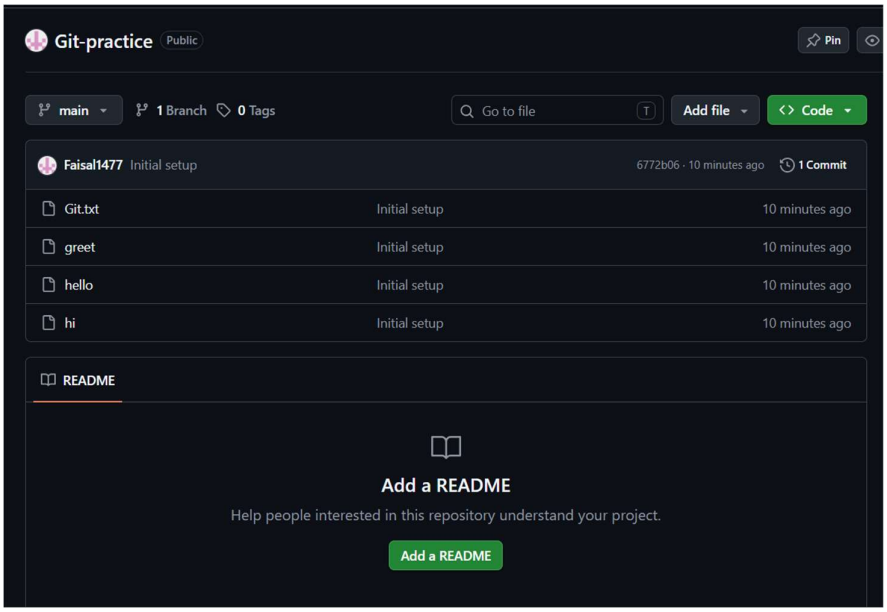
```
Faisal@LAPTOP-4DO8SS4E MINGW64 /d/Git (main)
$ git remote add origin https://github.com/Faisal1477/Git-practice.git

Faisal@LAPTOP-4DO8SS4E MINGW64 /d/Git (main)
$ git remote -v
origin  https://github.com/Faisal1477/Git-practice.git (fetch)
origin  https://github.com/Faisal1477/Git-practice.git (push)
```

Git push :- To push the commit files to GitHub we use push command. Here -u is used to form link between local repo to remote repo

```
Faisal@LAPTOP-4DO8SS4E MINGW64 /d/Git (main)
$ git push -u origin main
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 12 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (4/4), 276 bytes | 276.00 KiB/s, done.
Total 4 (delta 0), reused 0 (delta 0), pack-reused 0
To https://github.com/Faisal1477/Git-practice.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

After pushing it we can see our files in the GitHub repositories



Git Revert : - It is used to revert the done commits. We need Commit Id for this command

```
Faisal@LAPTOP-4DO8SS4E MINGW64 /d/Git (main)
$ git log
commit caddaf912d4f098bfbf6dcc3f0c8ec3f59e955ae (HEAD -> main, origin/main)
Author: Faisal1477 <mohdfaisal1477@gmail.com>
Date:   Thu Oct 22 16:45:15 2026 +0530

    Removed hi file

commit 6772b069214ebedebbfcb2bc85a199cf1df76124
Author: Faisal1477 <mohdfaisal1477@gmail.com>
Date:   Thu Oct 22 16:17:18 2026 +0530

    Initial setup
```

We use first 5 characters of the id with revert command. It will remove the changes done in that commit.

```
Faisal@LAPTOP-4DO8SS4E MINGW64 /d/Git (main)
$ git revert cadda
[main 28aaf5c] Revert "Removed hi file"
 1 file changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 hi

Faisal@LAPTOP-4DO8SS4E MINGW64 /d/Git (main)
$ ls
Git.txt  greet  hello  hi
```

Git reset :- It is used to reset the staging of the file from add command. We can unstage it for single file with name or without giving name it will reset all the files.

```
Faisal@LAPTOP-4DO8SS4E MINGW64 /d/Git (main)
$ git add .

Faisal@LAPTOP-4DO8SS4E MINGW64 /d/Git (main)
$ git status
On branch main
Your branch is ahead of 'origin/main' by 1 commit.
  (use "git push" to publish your local commits)

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        new file:   Addfile
```

```
Faisal@LAPTOP-4DO8SS4E MINGW64 /d/Git (main)
$ git reset

Faisal@LAPTOP-4DO8SS4E MINGW64 /d/Git (main)
$ git status
On branch main
Your branch is ahead of 'origin/main' by 1 commit.
  (use "git push" to publish your local commits)

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        Addfile

nothing added to commit but untracked files present (use "git add" to track)
```

Git Clone:- This command is used to pull the files from remote repo to local machine. But whenever we clone all files will be clone including the ones we have, so the disk utilization will increase with.

It should be only used when we don't have any files of that project.

```
Faisal@LAPTOP-4DO8SS4E MINGW64 /d/clone
$ git clone https://github.com/Faisal1477/Git-practice.git
Cloning into 'Git-practice'...
remote: Enumerating objects: 9, done.
remote: Counting objects: 100% (9/9), done.
remote: Compressing objects: 100% (5/5), done.
remote: Total 9 (delta 2), reused 5 (delta 1), pack-reused 0 (from 0)
Receiving objects: 100% (9/9), done.
Resolving deltas: 100% (2/2), done.

Faisal@LAPTOP-4DO8SS4E MINGW64 /d/clone
$ ls
Git-practice/
```

Git Pull : -This is the easy way for clone it take the files which are not present in the project folder or new files created by others.

We can use link or just pull to update the files from the Repo. It has a pull command from another branches.

```
Faisal@LAPTOP-4DO8SS4E MINGW64 /d/Git (main)
$ ls
Addfile  Git.txt  greet  hello  hi

Faisal@LAPTOP-4DO8SS4E MINGW64 /d/Git (main)
$ git pull
remote: Enumerating objects: 4, done.
remote: Counting objects: 100% (4/4), done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (3/3), 994 bytes | 142.00 KiB/s, done.
From https://github.com/Faisal1477/Git-practice
   caddaf9..aeb6302  main       -> origin/main
Merge made by the 'ort' strategy.
 Text | 1 +
 1 file changed, 1 insertion(+)
 create mode 100644 Text

Faisal@LAPTOP-4DO8SS4E MINGW64 /d/Git (main)
$ ls
Addfile  Git.txt  Text  greet  hello  hi
```